



SDA WEEK 7

ACTIVITY DIAGRAMS

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

LEARNING OBJECTIVES

By the end of this week, you will be able to:

1. Understand the purpose and components of an activity diagram.
2. Model sequential, parallel, and conditional flows.
3. Use advanced features like object flow, swimlanes, and structured activities.
4. Apply this knowledge to model business processes and complex operations.

WHAT IS AN ACTIVITY?

- An activity represents a task to be done either by a human or a computer.
- It can be a method in a class or any task in a process.
- Activities can be arranged sequentially (one after another) or in parallel (multiple activities concurrently).

WHAT IS AN ACTIVITY DIAGRAM?

An Activity Diagram represents the dynamics of a system. It shows the workflow—the flow of control from one activity to another.

Think of it as an advanced, more powerful version of a classic flow chart. While flowcharts are typically limited to sequential steps, activity diagrams can explicitly model parallel activities, which is crucial for modern systems.

WHAT IS AN ACTIVITY DIAGRAM?

An Activity Diagram represents the dynamics of a system. It shows the workflow—the flow of control from one activity to another.

Think of it as an advanced, more powerful version of a classic flow chart. While flowcharts are typically limited to sequential steps, activity diagrams can explicitly model parallel activities, which is crucial for modern systems.

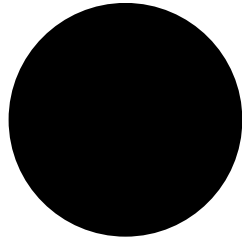
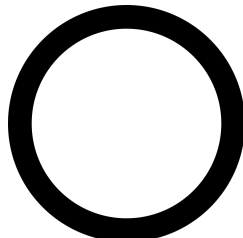
- Flowcharts = sequential.
- Activity Diagrams = sequential + parallel + object flow.

PURPOSE

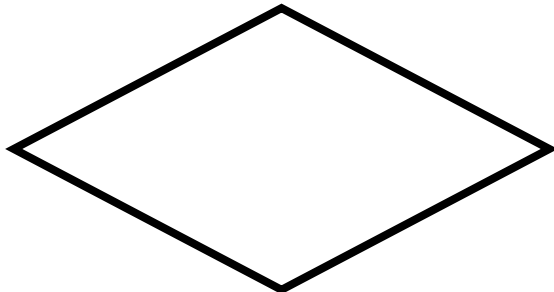
Activity Diagrams visualize system workflows:

- Show flow of control from one activity to another.
- Represent parallel activities.
- Illustrate alternate paths.
- Modeling the detailed flow of a use case, as a use case is essentially a series of activities
- Documenting business processes across an organization.

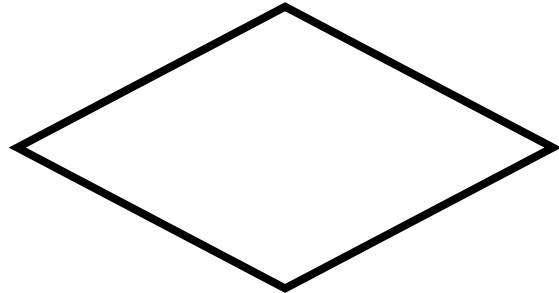
CORE SYMBOLS & NOTATIONS

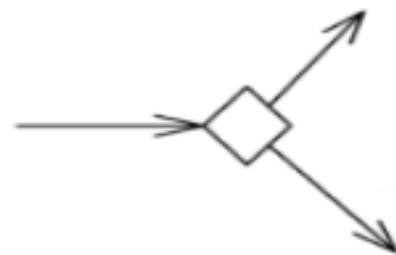
Name	Symbol	Description
Start Node		This is where the workflow begins. It's represented by a solid black circle . Every diagram must have one.
Stop Marker		This is a black circle with a concentric ring . It represents the end of the entire workflow. When this node is reached, all activity stops.
Activity		This is a rectangle with rounded corners. Inside, we write a brief description of the task being performed, like 'Validate PIN' or 'Dispatch Order'. This is the fundamental building block

CORE SYMBOLS & NOTATIONS

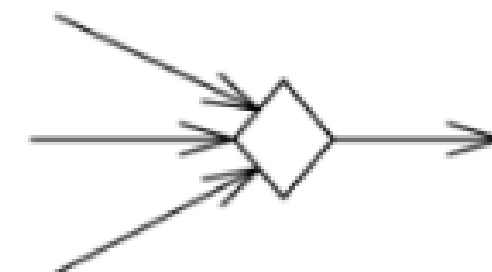
Name	Symbol	Description
Control flow or Transition		This is a simple arrow connecting activities. It shows the direction of the flow. When one activity finishes, control immediately passes to the next activity via the transition.
Decision Node or Branching		This is a diamond . It represents a choice in the workflow. It has one incoming arrow and two or more outgoing arrows . Each outgoing arrow must have a guard condition in square brackets, like [PIN valid] or [PIN invalid]. These conditions must be mutually exclusive

CORE SYMBOLS & NOTATIONS

Name	Symbol	Description
Merge Node		This is another diamond . It is used to combine alternate flows back into a single flow. It has two or more incoming arrows and one outgoing arrow. It's the opposite of a decision node.



Condition Node



Merge Node

CORE SYMBOLS & NOTATIONS

Name	Symbol	Description
Final flow or Final node		This is a circle with an X inside. It is used to terminate a specific flow within a larger, possibly parallel, activity. It does not stop the entire diagram, only the path it is on.

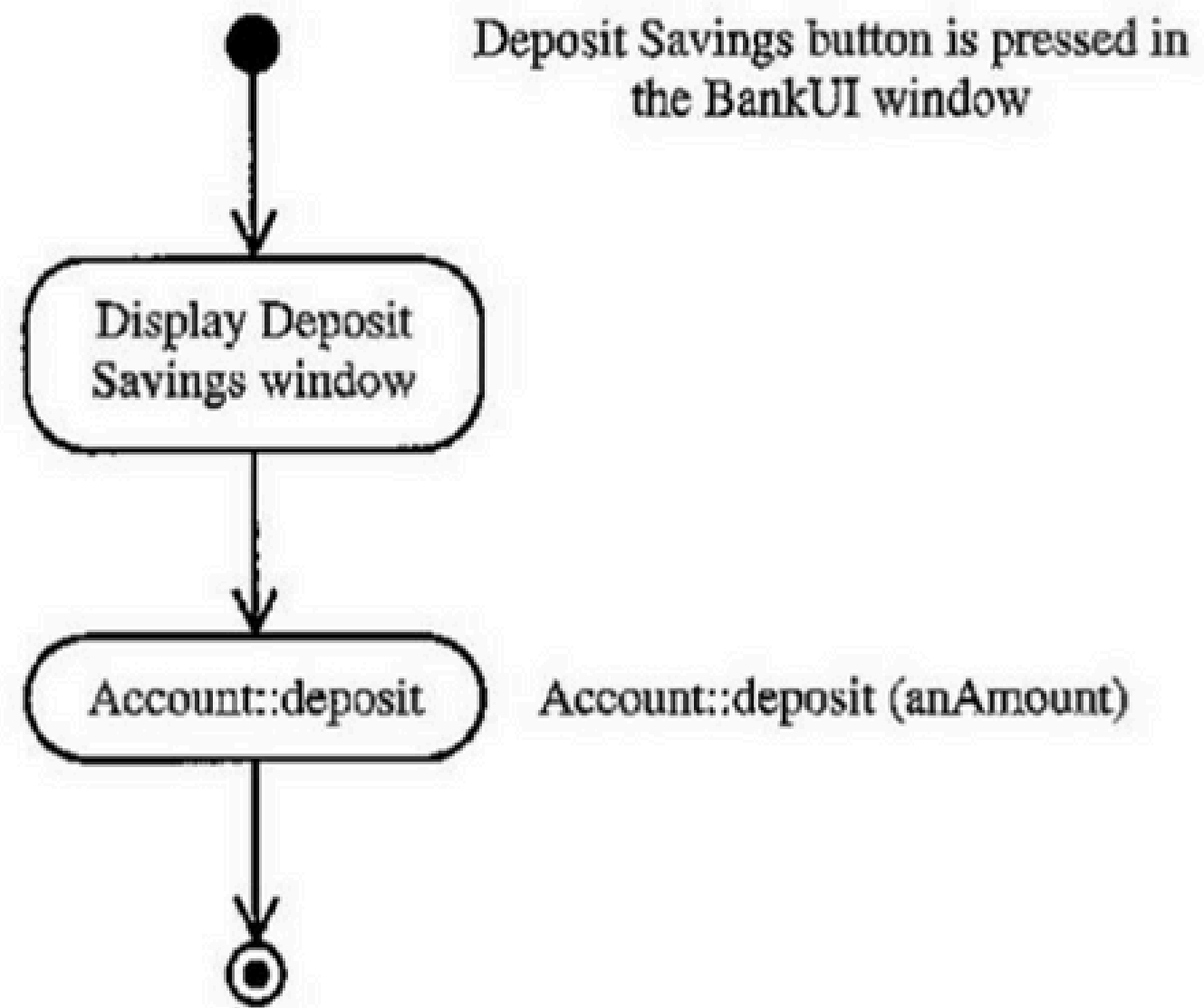
EXAMPLES

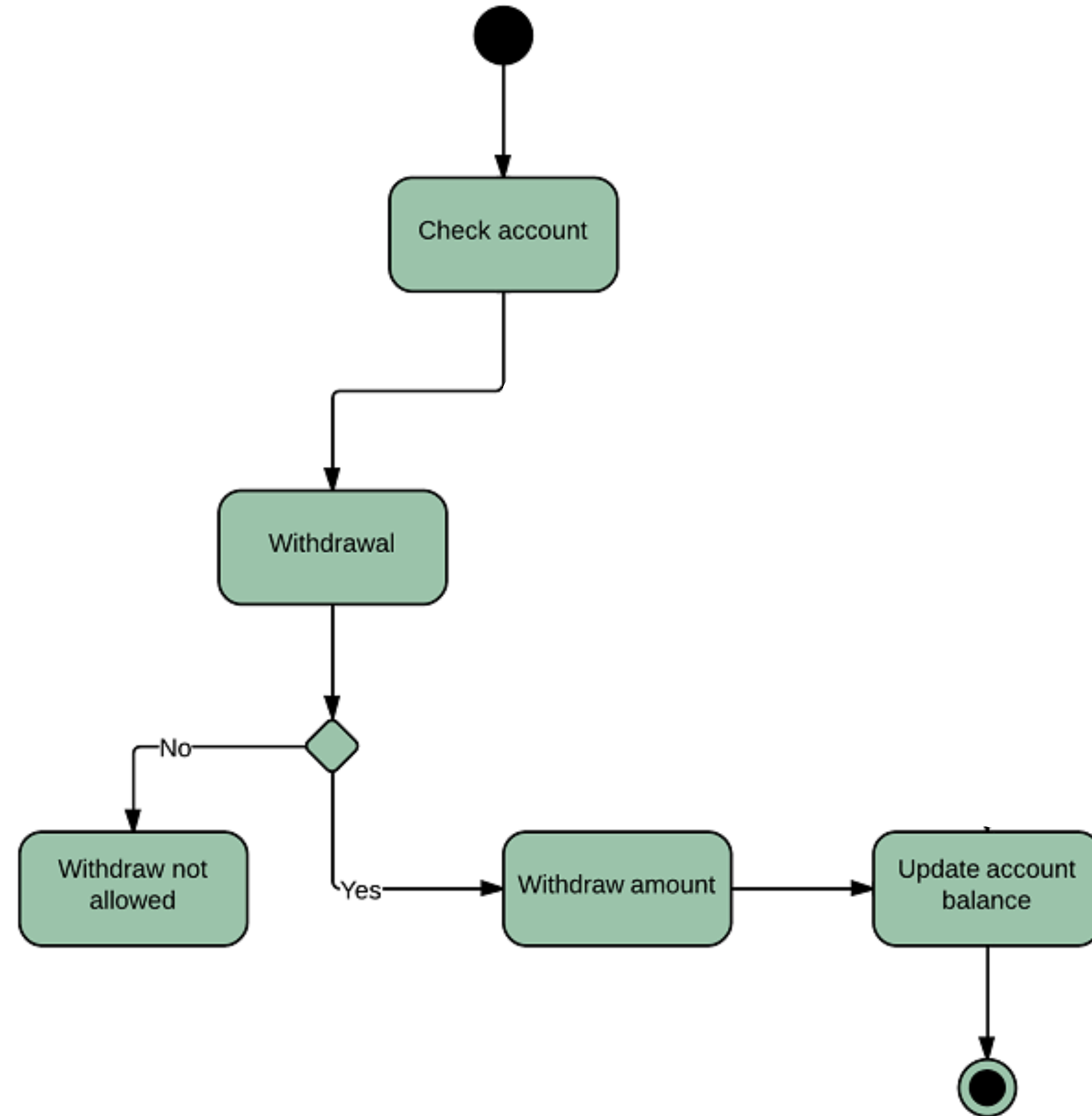
Let's look at a simple example: 'Processing a Deposit.' The flow is:

1. Start.
2. 'Deposit Savings' button is pressed.
3. 'Display Deposit Savings Window.'
4. Execute the `Account::deposit(amount)` operation.
5. End.

EXAMPLES

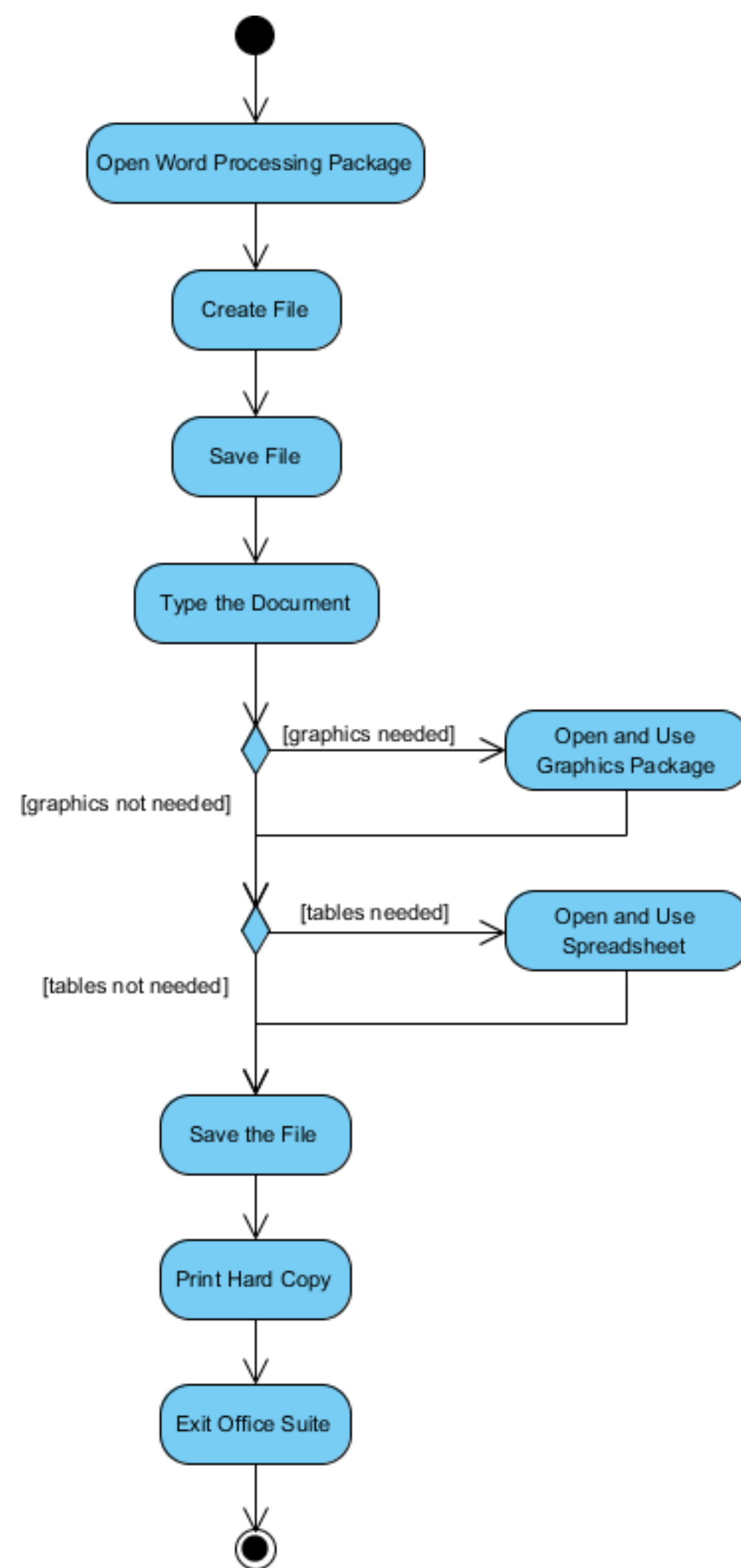
Activity diagram for processing a deposit to a savings account.

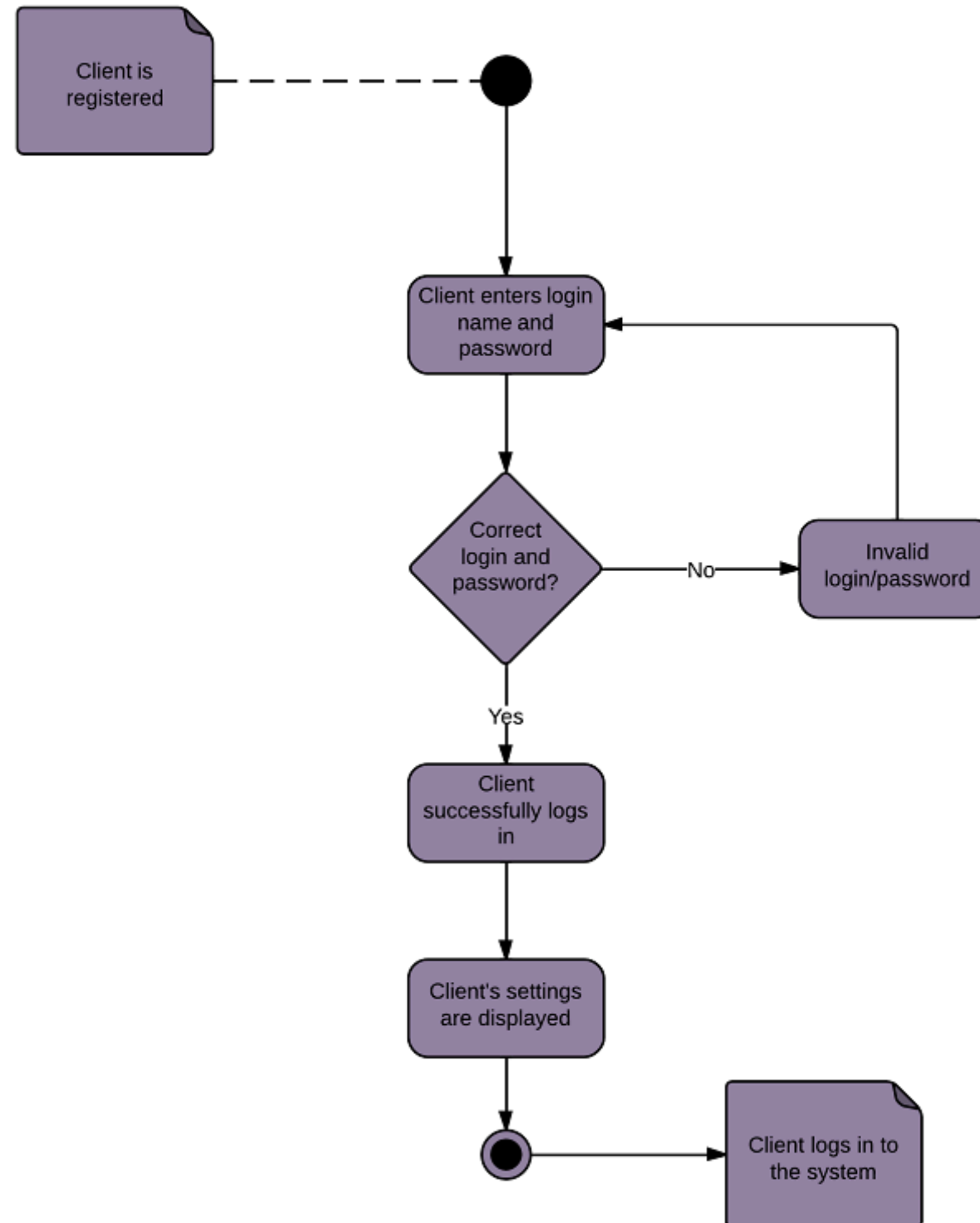




EXAMPLES

- Open the word processing package.
- Create a file.
- Save the file under a unique name within its directory.
- Type the document.
- If graphics are necessary, open the graphics package, create the graphics, and paste the graphics into the document.
- If a spreadsheet is necessary, open the spreadsheet package, create the spreadsheet, and paste the spreadsheet into the document.
- Save the file.
- Print a hard copy of the document.
- Exit the word processing package.





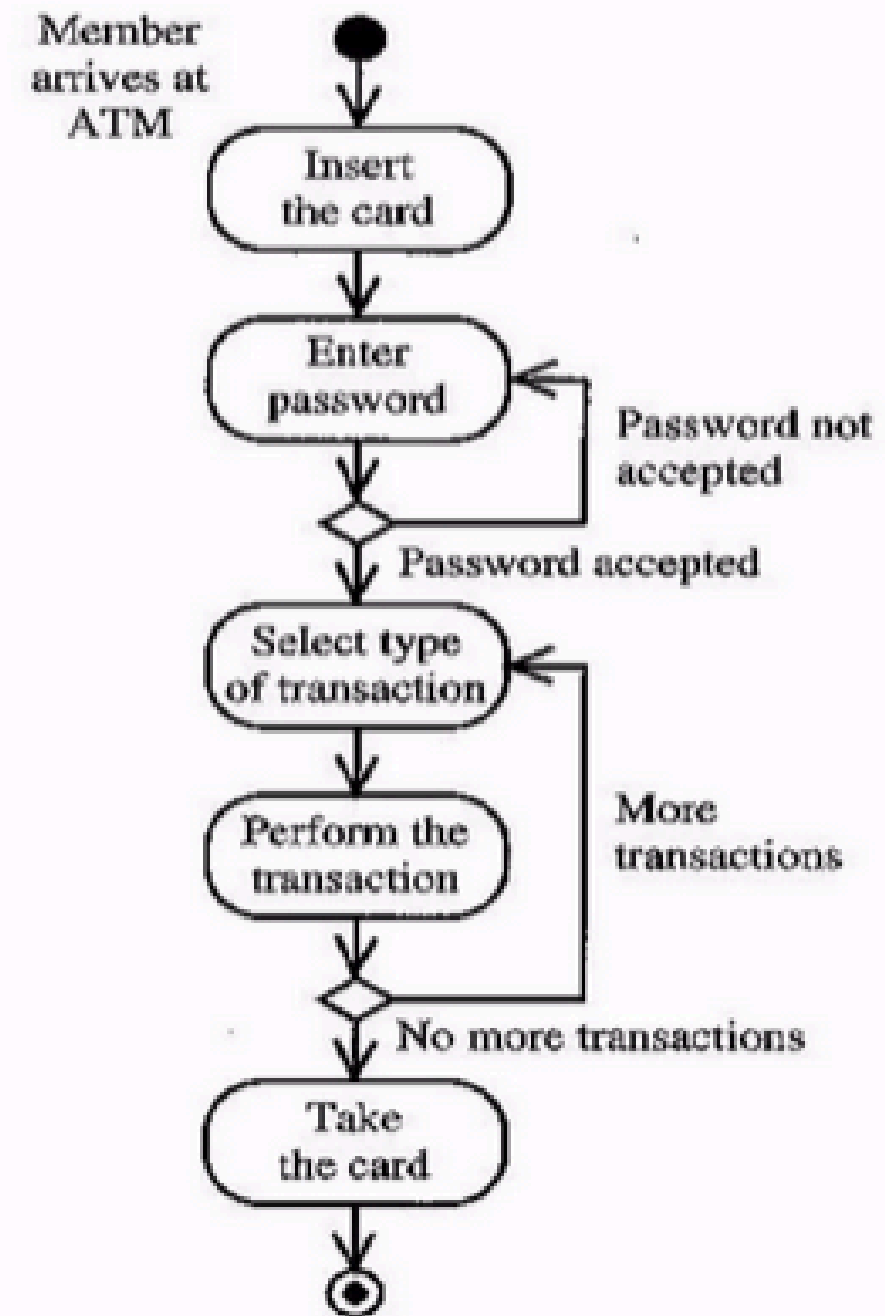


FIGURE 6-8
Activities involved in an ATM transaction.

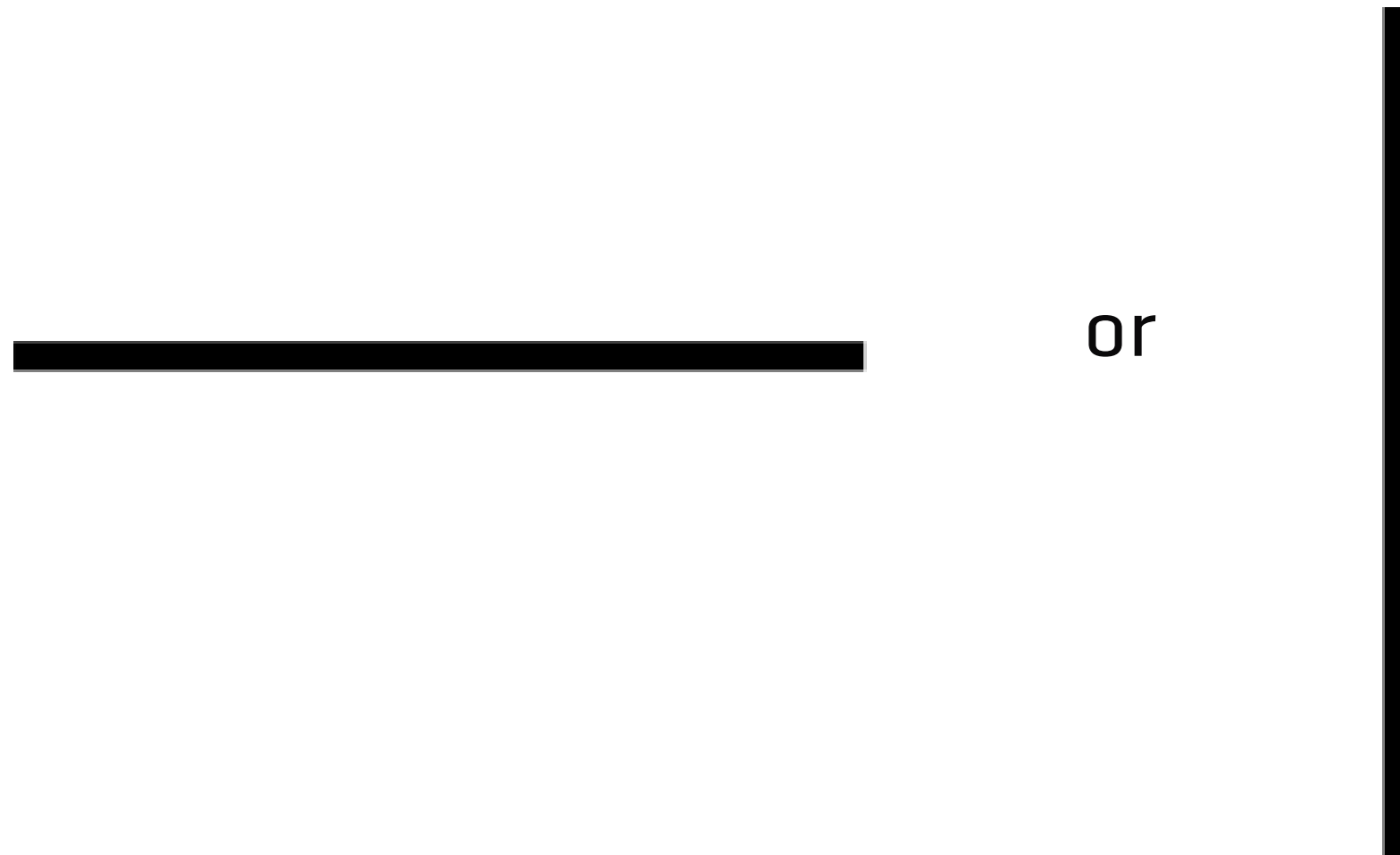


INTRODUCTION TO PARALLELISM

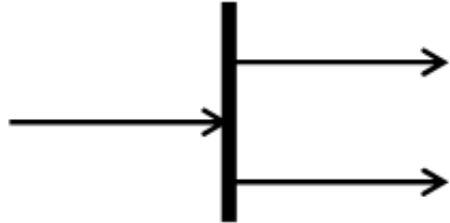
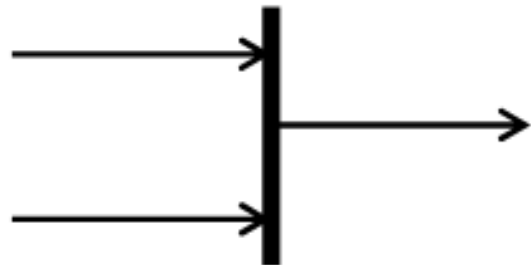
- So far we have seen how to model a linear, sequential flow of activities.
- But what about when things happen at the same time?
- For example, when you order a product online, the warehouse often 'Fills the Order' at the same time as the billing department 'Sends the Invoice'. These are concurrent activities.

MODELING CONCURRENCY

Synchronization bar: To model parallel flows, we use two powerful symbols: the Fork and the Join. Both are represented as a thick horizontal or vertical line, often called a synchronization bar.



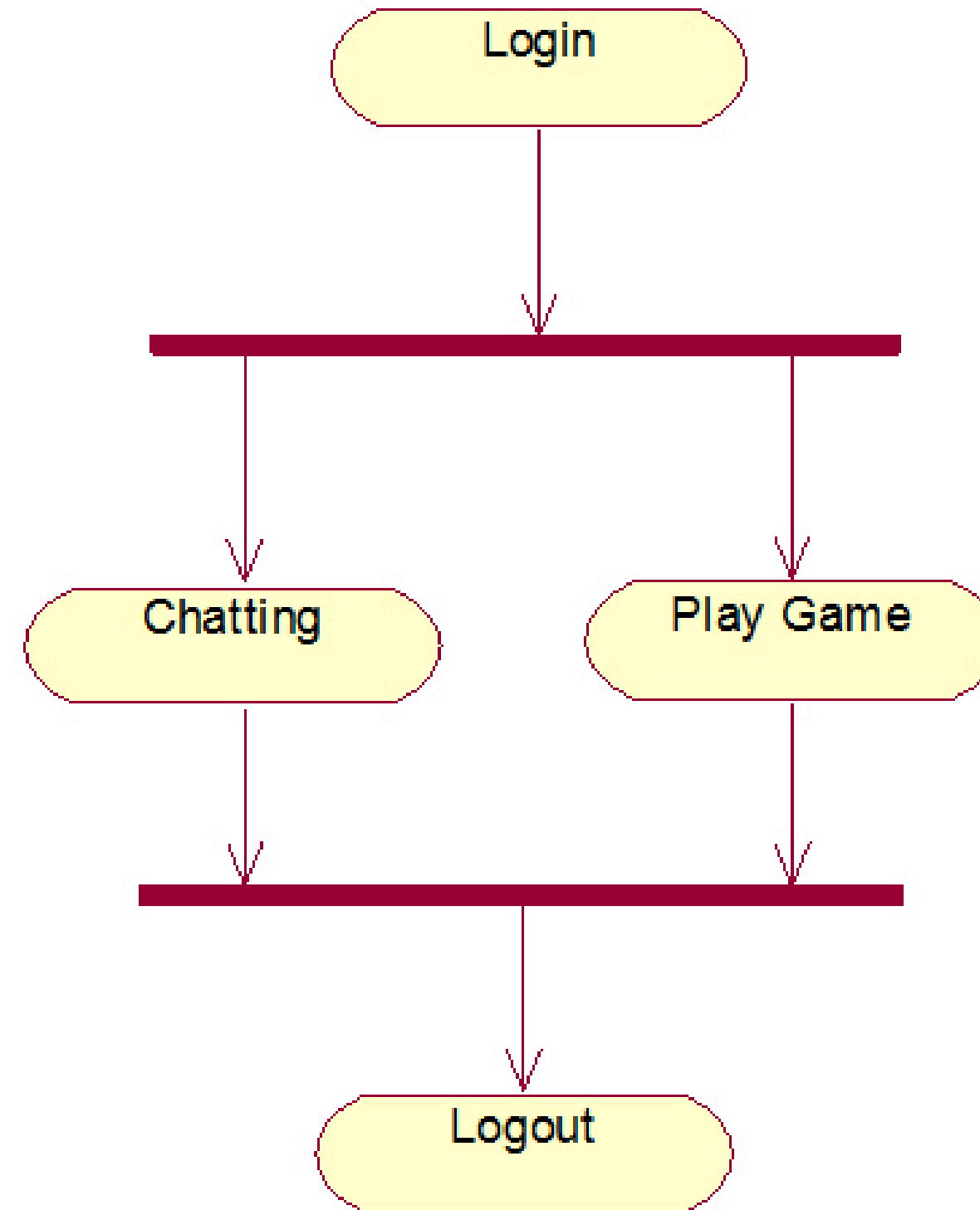
FORK & JOIN

Name	Symbol	Description
Fork		A fork has one incoming transition and two or more outgoing transitions . When the incoming flow reaches the fork, it splits into multiple, independent concurrent flows. For example, after 'Receive Order,' we can fork into 'Fill Order' and 'Send Invoice'. These now happen simultaneously.
Join		A join has two or more incoming transitions and one outgoing transition . The flow after the join will not execute until all incoming flows have reached the join. For example, we cannot 'Close Order' until both 'Ship Order' and 'Receive Payment' are complete. The join synchronizes the parallel threads.

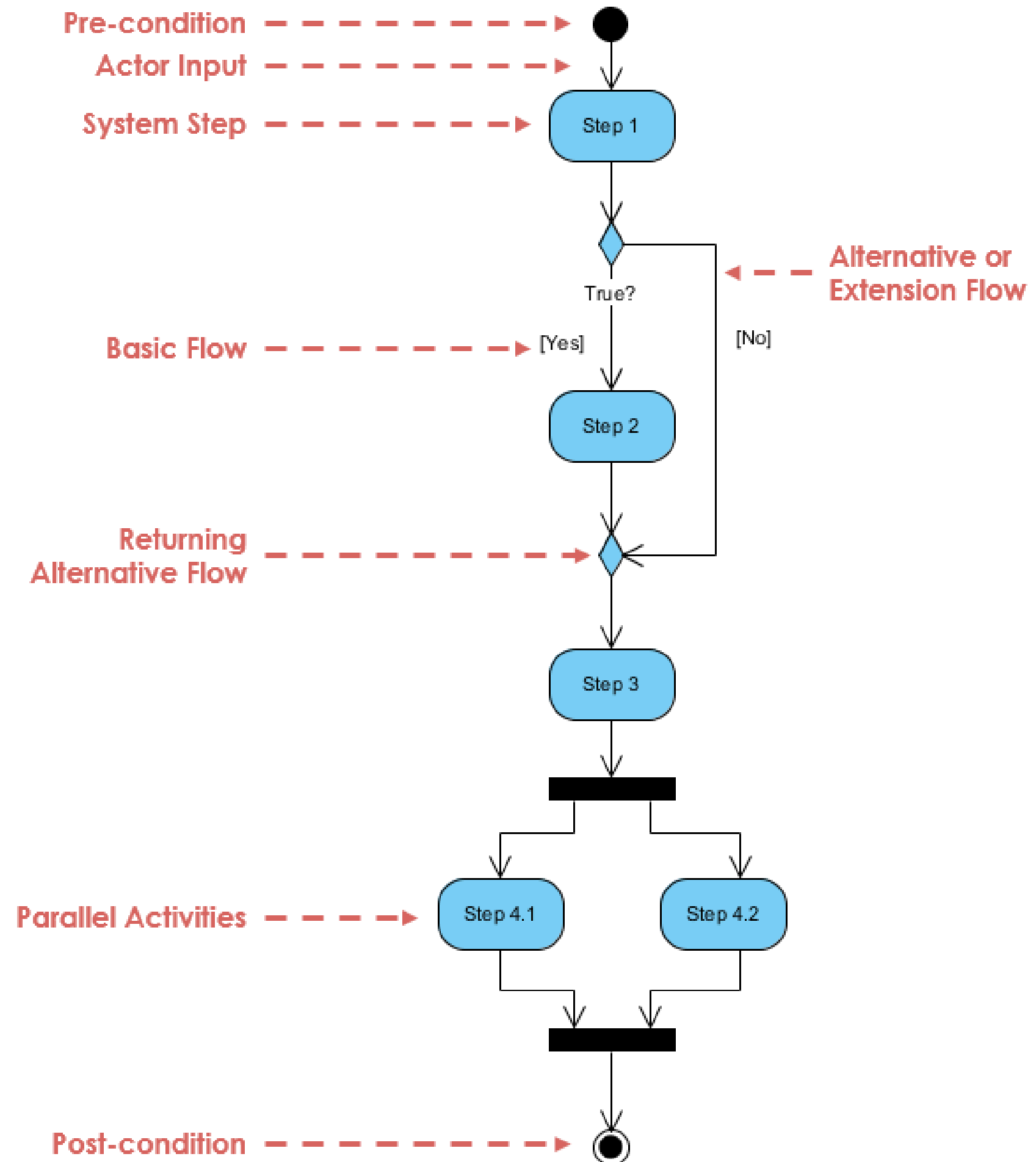
EXAMPLES

Design an activity diagram for a **course registration system**:

- Student logs in.
- If credentials invalid → show error message.
- If valid → check seat availability.
- If seat available, system forks into:
 - Fee payment.
 - Academic approval.
- Both must complete before joining into → Confirm Registration.



Basic Activity diagram to visualize branching, Fork and join

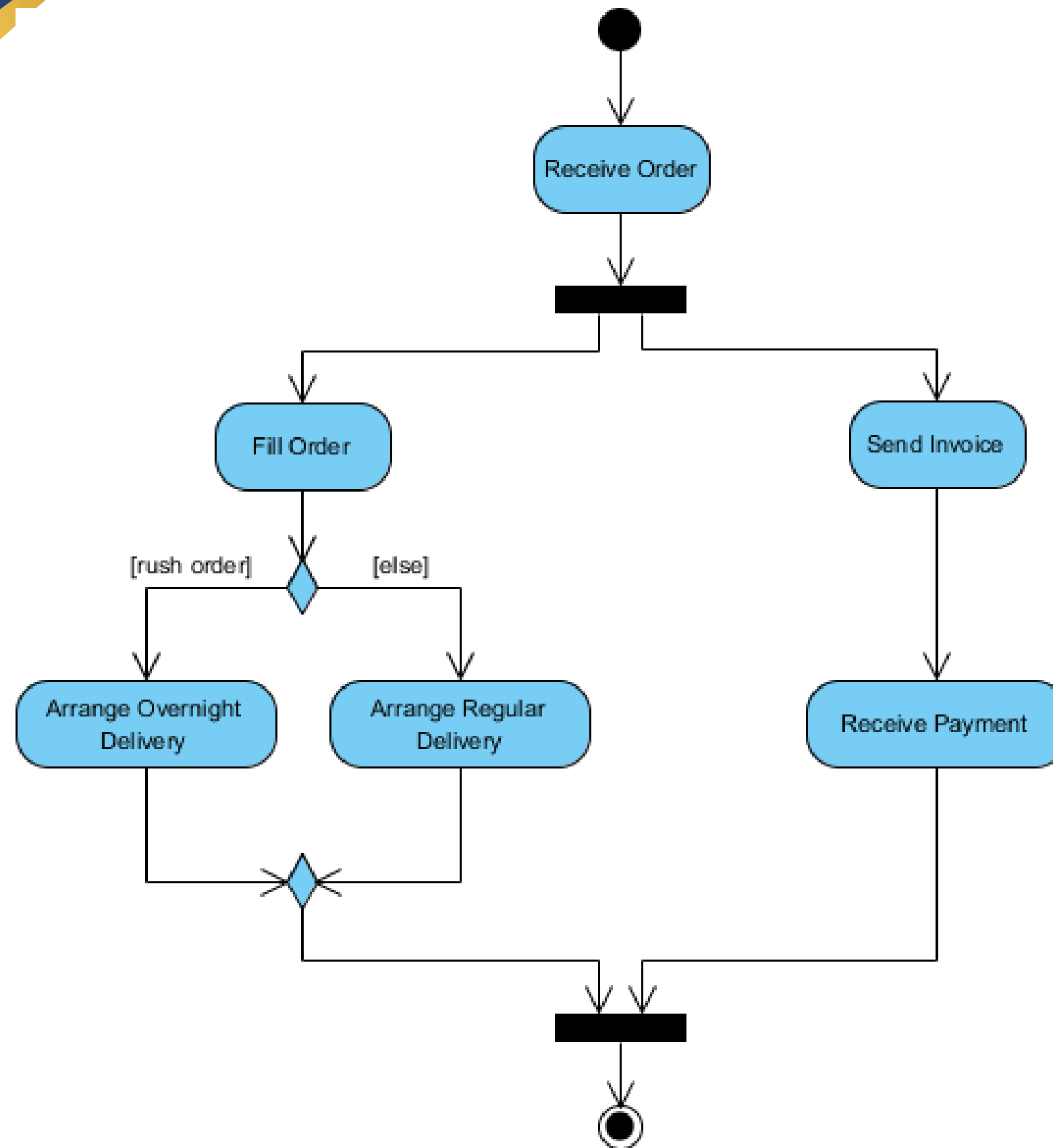


Process Order - Problem Description

Once the order is received, the activities split into two parallel sets of activities. One side fills and sends the order while the other handles the billing.

On the Fill Order side, the method of delivery is decided conditionally. Depending on the condition either the Overnight Delivery activity or the Regular Delivery activity is performed.

Finally the parallel activities combine to close the order.



SWIMLANE

In activity diagrams, the activity states can be divided into logical groups, each group representing the object responsible for the activities. In UML, each group is known as a swimlane because, visually, each group is divided from its neighbor by a vertical solid line as shown in the below figure.

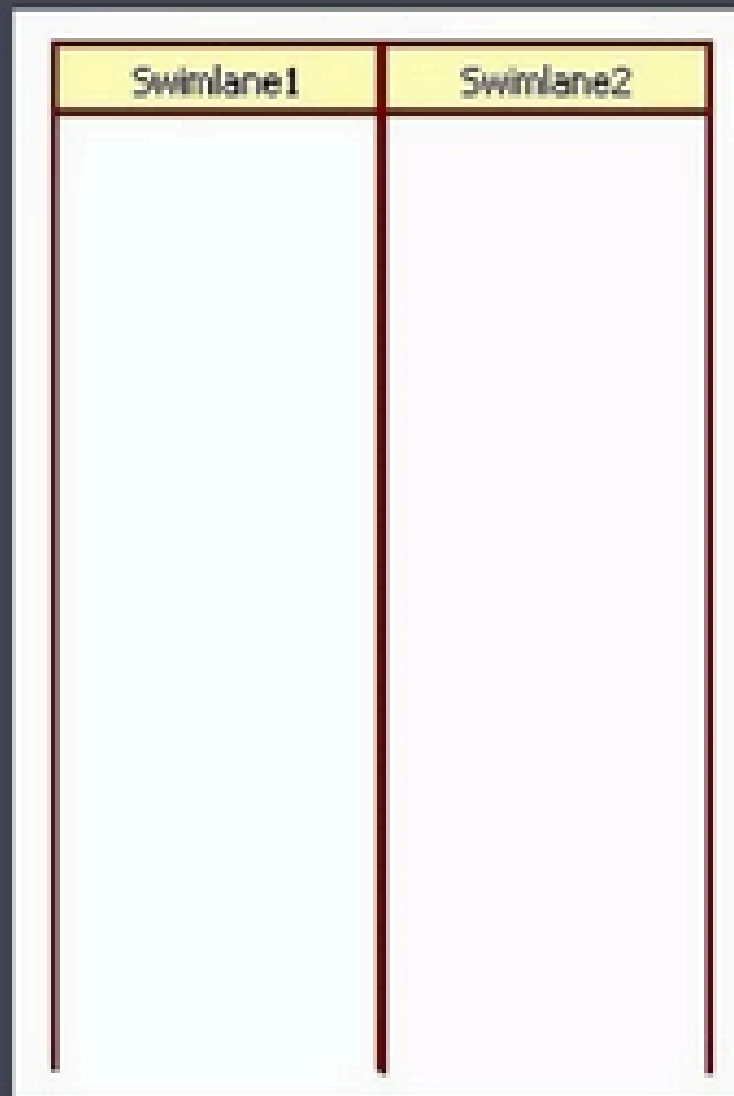
Each swimlane has its own name. In an activity diagram partitioned into swimlanes, every activity belongs to exactly one swimlane, but transitions may cross lanes.

SWIMLANE

Swimlane visually distinguishes job sharing and responsibilities for sub-processes of a business process. Swim lanes may be arranged either horizontally or vertically.

- Each swimlane has a name unique within its diagram
- Each swimlane may represent some real-world entity
- Each swimlane may be implemented by one or more classes
- Every activity belongs to exactly one swimlane, but transitions may cross lanes

SWIMLANE

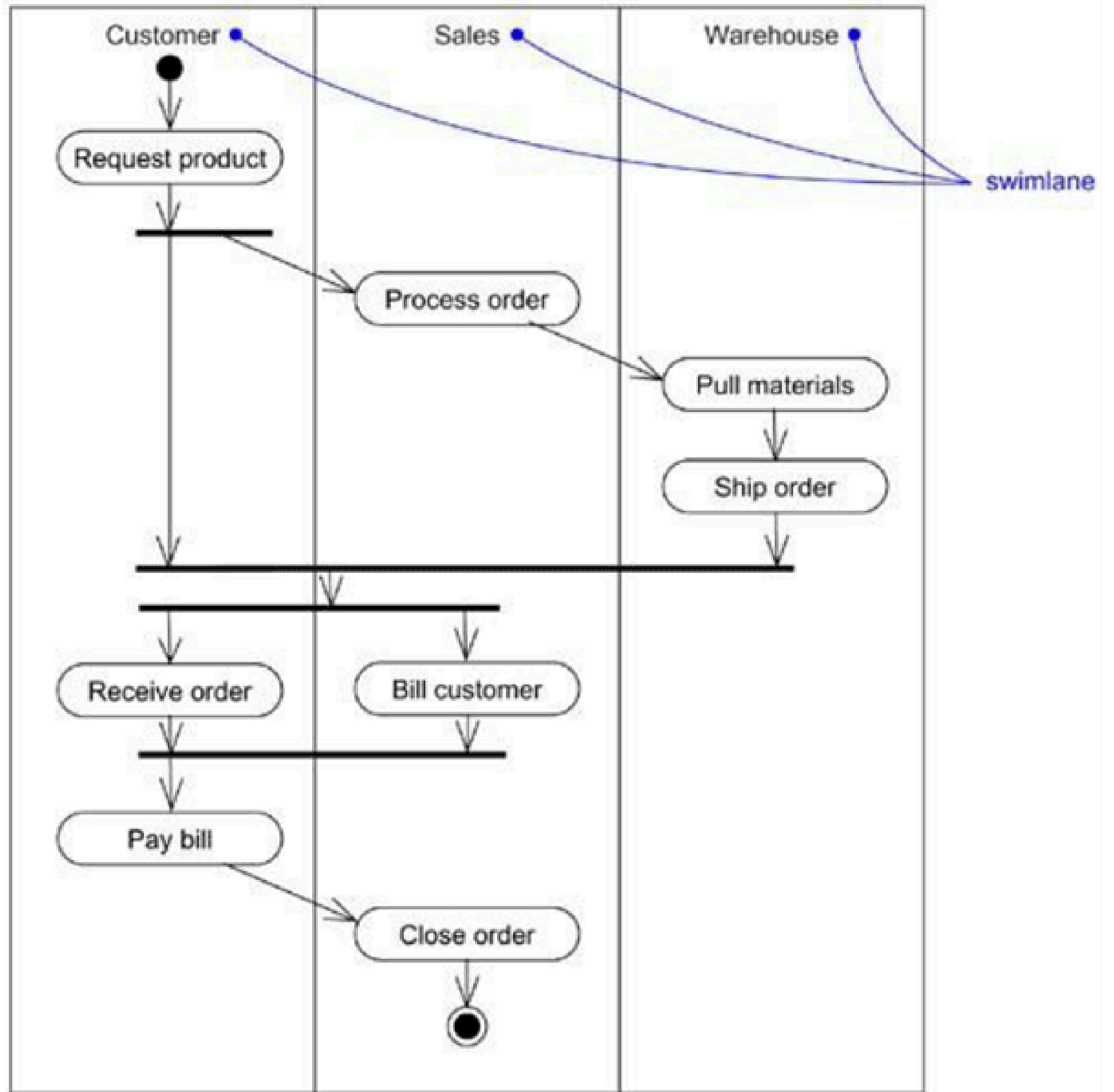


Vertical Swimlane



Horizontal Swimlane

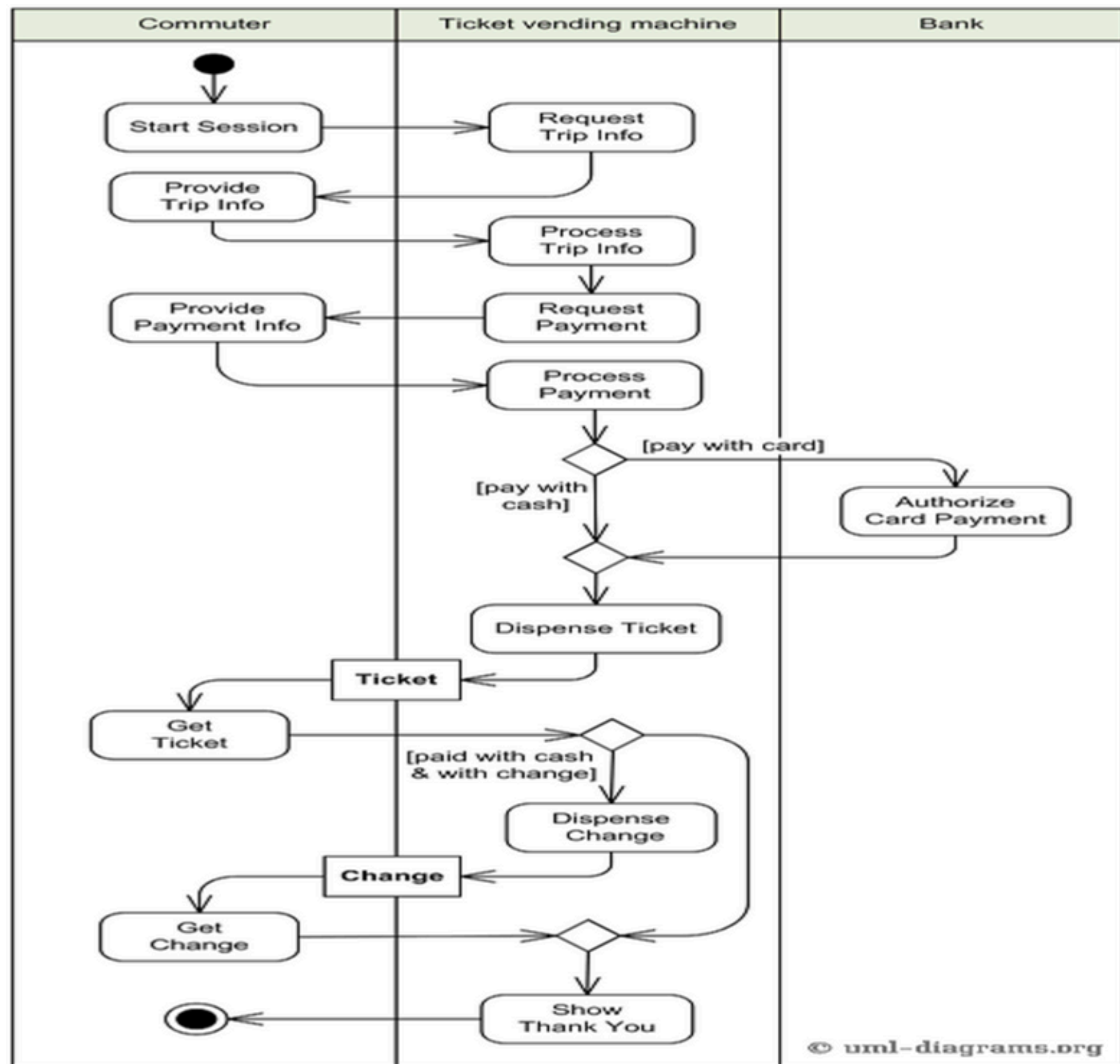
EXAMPLE 1



TICKET VENDING MACHINE

Activity is started by Commuter actor who needs to buy a ticket. Ticket vending machine will request trip information from Commuter. This information will include number and type of tickets, e.g. whether it is a monthly pass, one way or round ticket, route number, destination or zone number, etc.

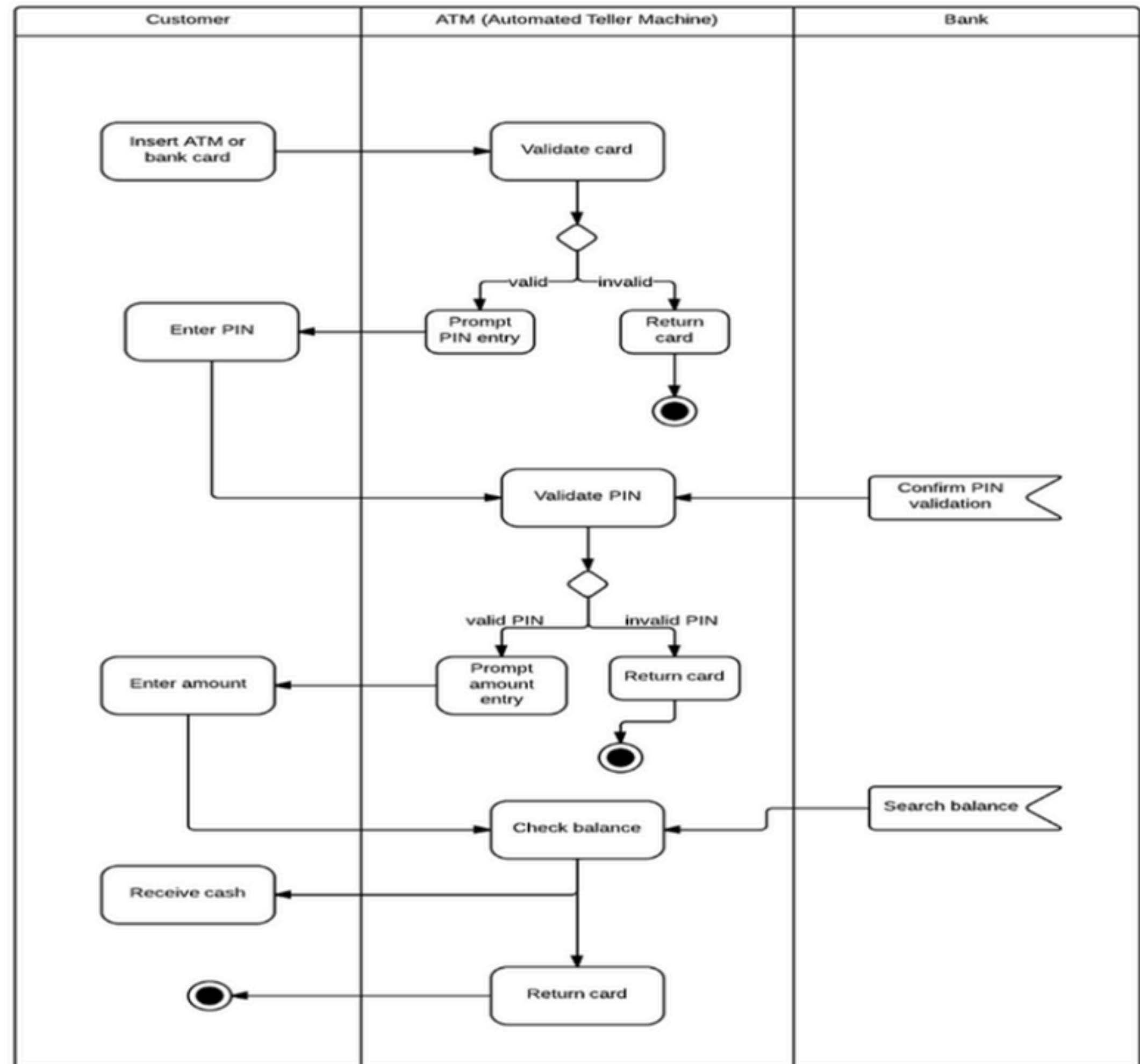
Based on the provided trip info ticket vending machine will calculate payment due and request payment options. Those options include payment by cash, or by credit or debit card. If payment by card was selected by Commuter, another actor, Bank will participate in the activity by authorizing the payment.



EXERCISE

A customer wants to draw money from his bank account. He enters his card into an ATM (automated teller machine). The ATM machine accepts the card and prompts for PIN number. The customer enters his PIN. The ATM (internally) retrieves the bank account number from the card. The ATM encrypts the PIN and the account number and sends it over to the bank. The bank verifies the encrypted Account and PIN number. Card is returned in case of invalid pin number. If the PIN number is correct, the ATM displays “Enter amount”, draws money from the bank account and prints the receipt.

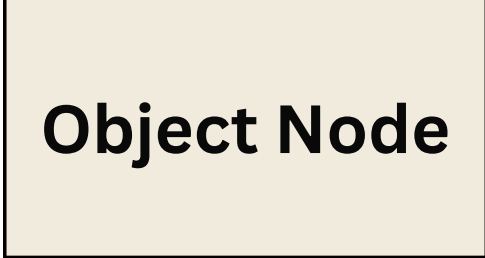
SOLUTION



OBJECT FLOW

Object Node:

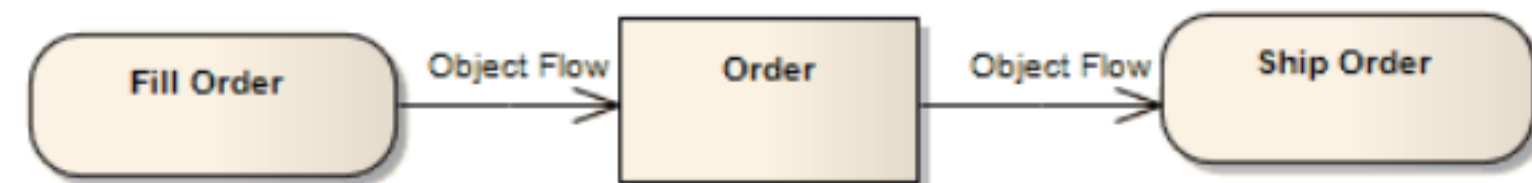
- Symbol: A rectangle.
- Meaning: Represents an instance of a classifier (e.g., an object of a class) that is present in the workflow at a specific point.
- Objects often change state between activities.



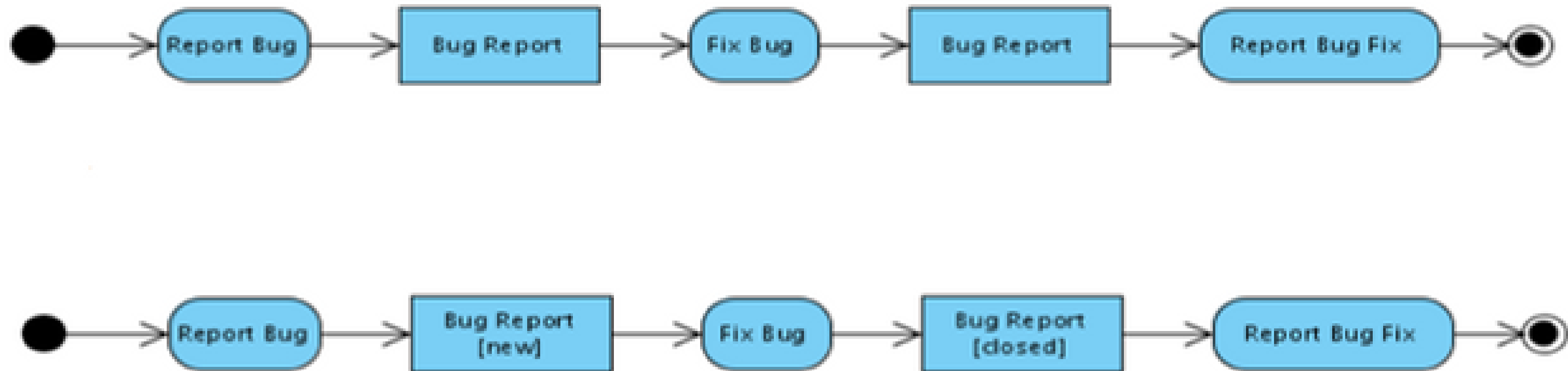
Object Node

Object Flow:

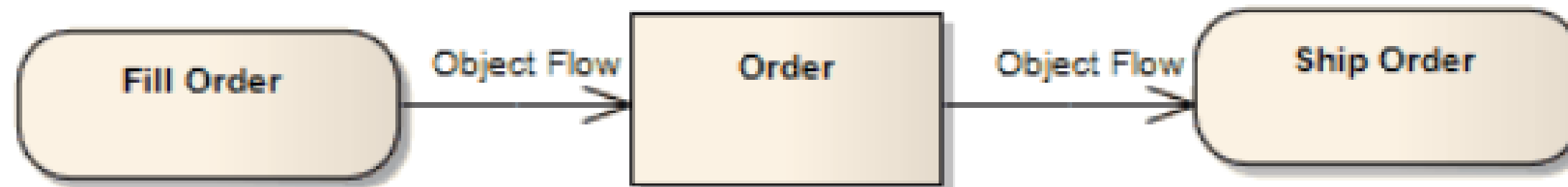
- Symbol: A dashed arrow.
- Meaning: Shows that an object is being passed from one activity to another. It plays the role of a data flow.



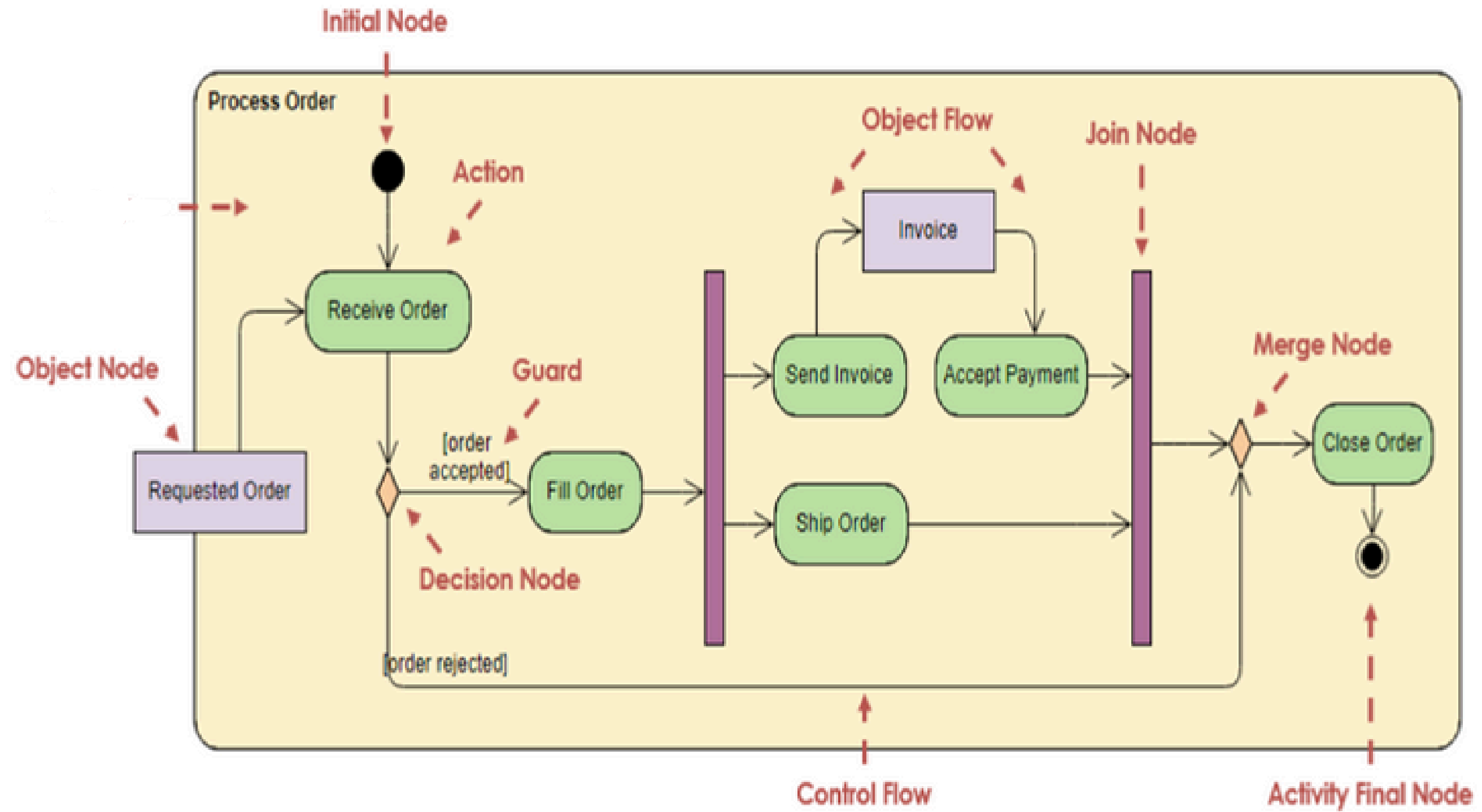
EXAMPLE: BUG FIXING WORKFLOW



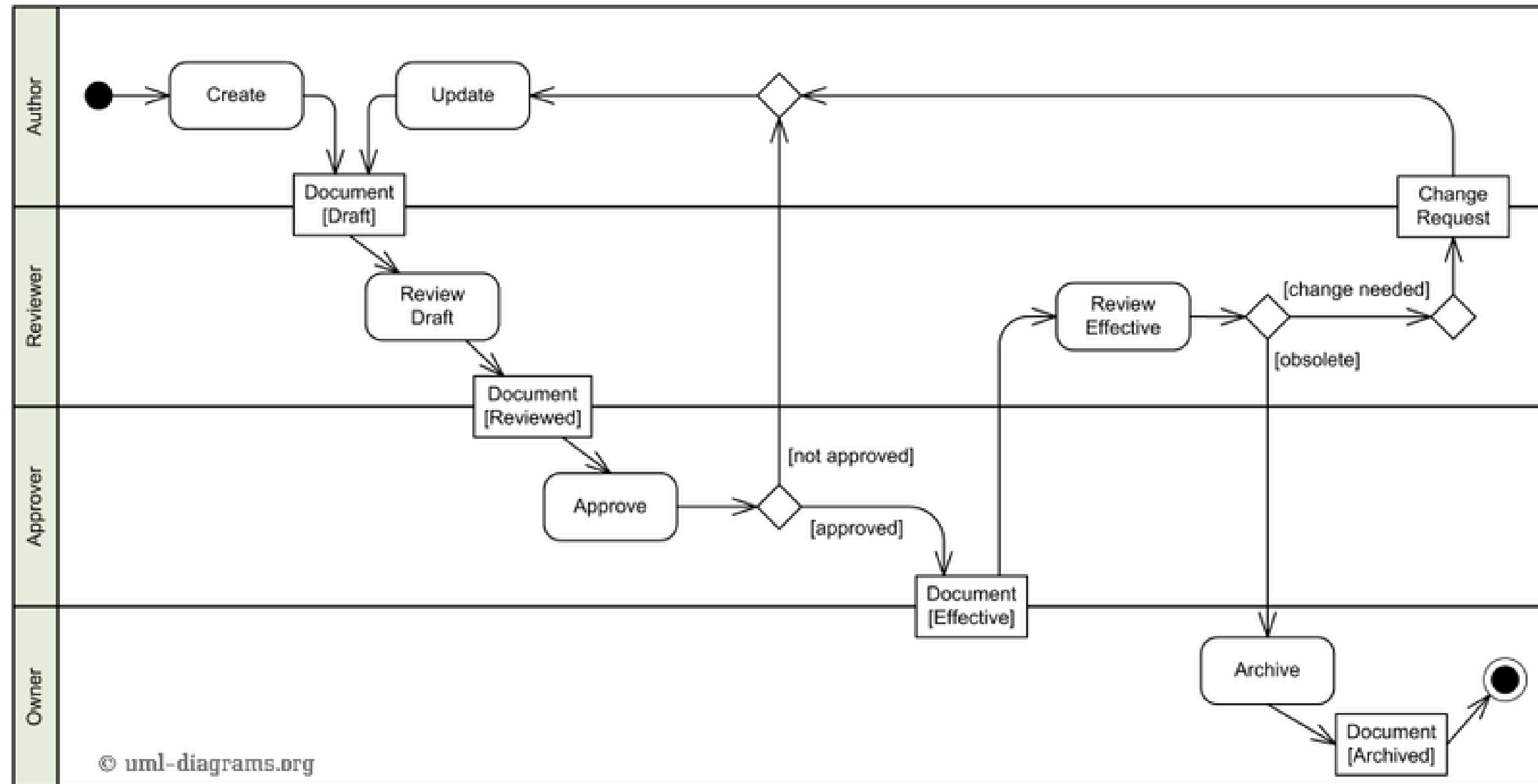
CONTROL FLOW VS OBJECT FLOW



EXAMPLE



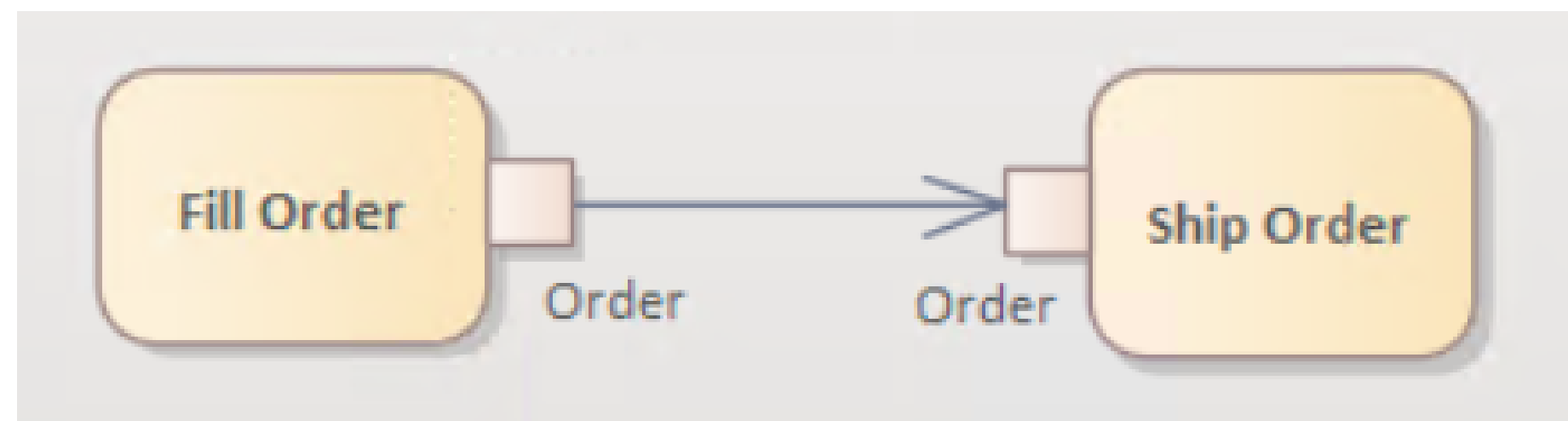
EXAMPLE: DOCUMENT MANAGEMENT



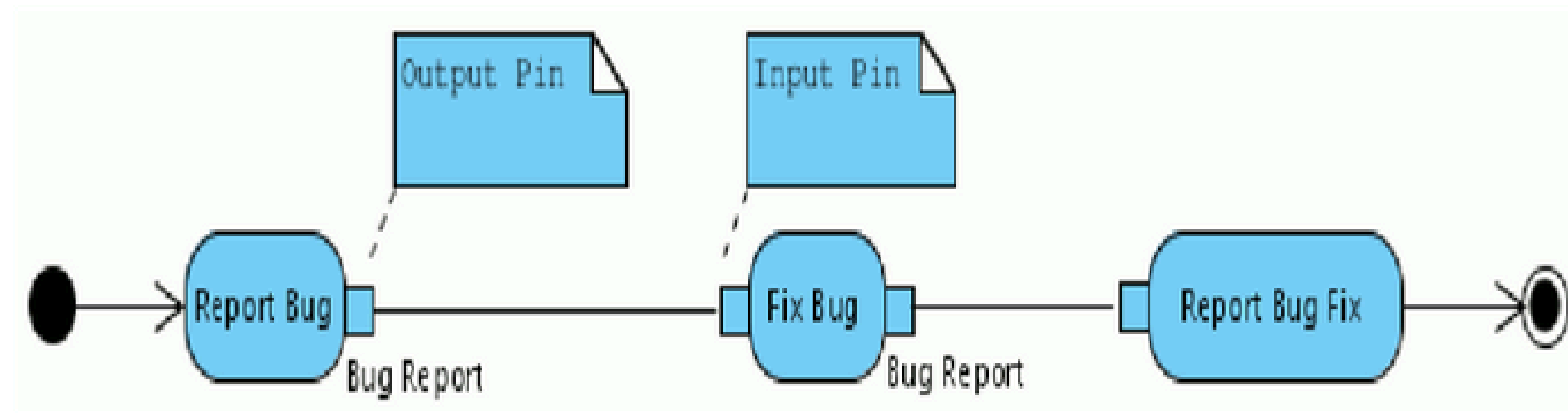
INPUT AND OUTPUT PINS

Instead of drawing explicit object nodes, we can use pins on the activities.

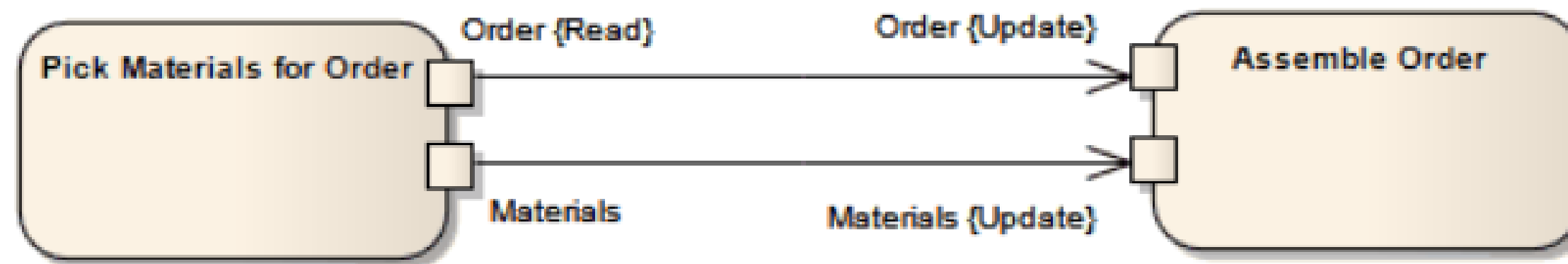
- Output Pin: A small square on the right of an activity where an object flows out.
- Input Pin: A small square on the left of an activity where an object flows in.



INPUT AND OUTPUT PINS

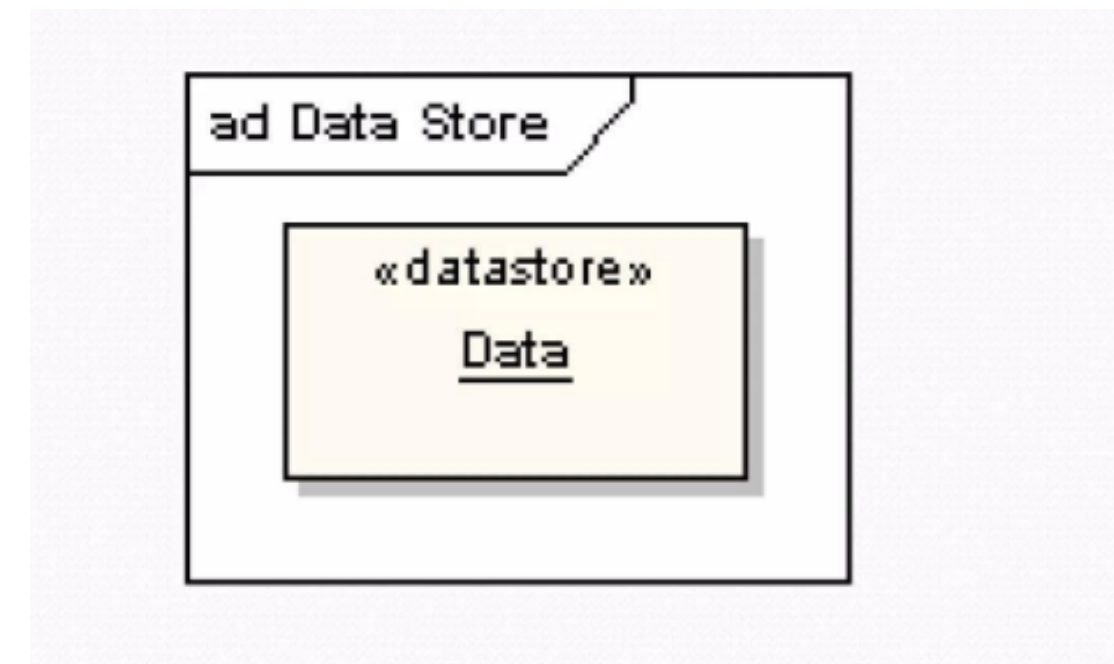


MULTIPLE OBJECT FLOWS



DATASTORE

What if the data isn't just passed from one activity to the next, but is stored for later use? For that, we have a Data Store. As shown in the figure, it's an object node with the «datastore» stereotype. Think of it as a database, a filing cabinet, or any persistent storage. Activities can read from and write to this node.



STRUCTURED ACTIVITY

A Structured Activity (<<structured>>) is a node that contains a subordinate, nested group of activities. It's used for managing complex synchronization.

A very important type of structured activity is the **Expansion Region**.

EXPANSION REGION

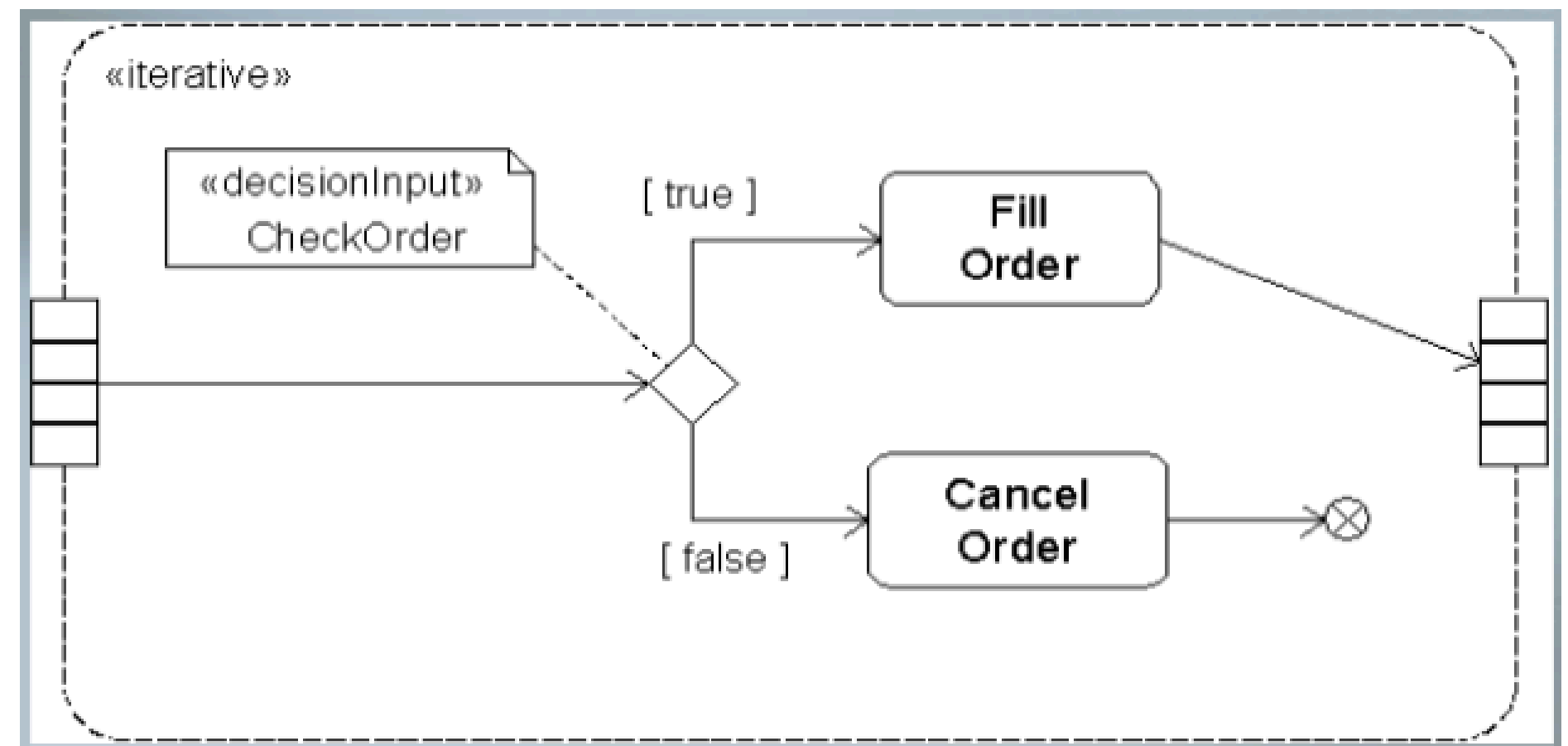
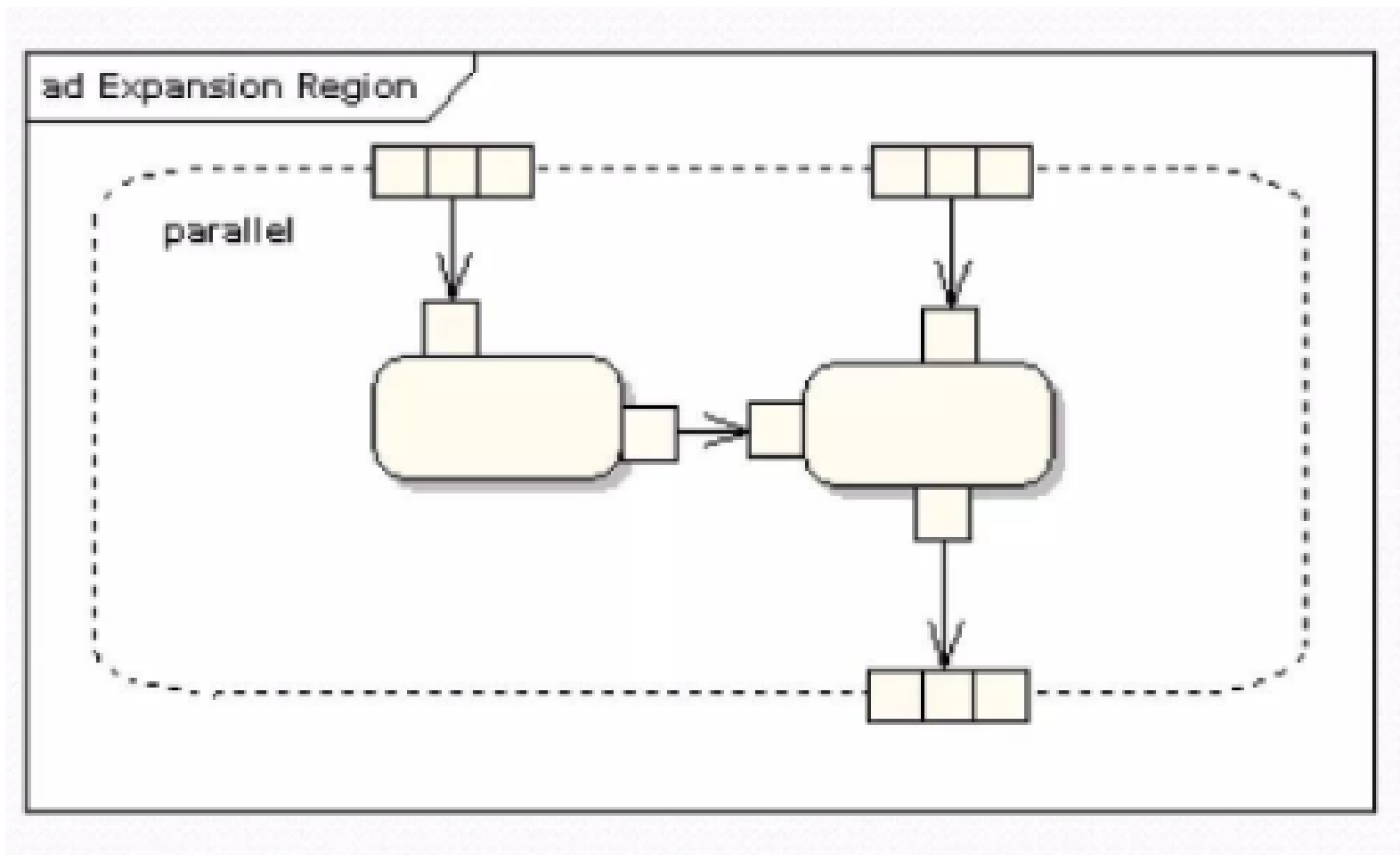
Shown with a **dashed rectangle**. It executes the activities inside it multiple times, once for each element in an input collection.



Execution Modes:

- **parallel:** All elements are processed at the same time.
- **iterative:** Elements are processed one after another, in sequence.
- **stream:** A single instance of the region processes elements as they arrive, like an assembly line.

EXPANSION REGION...



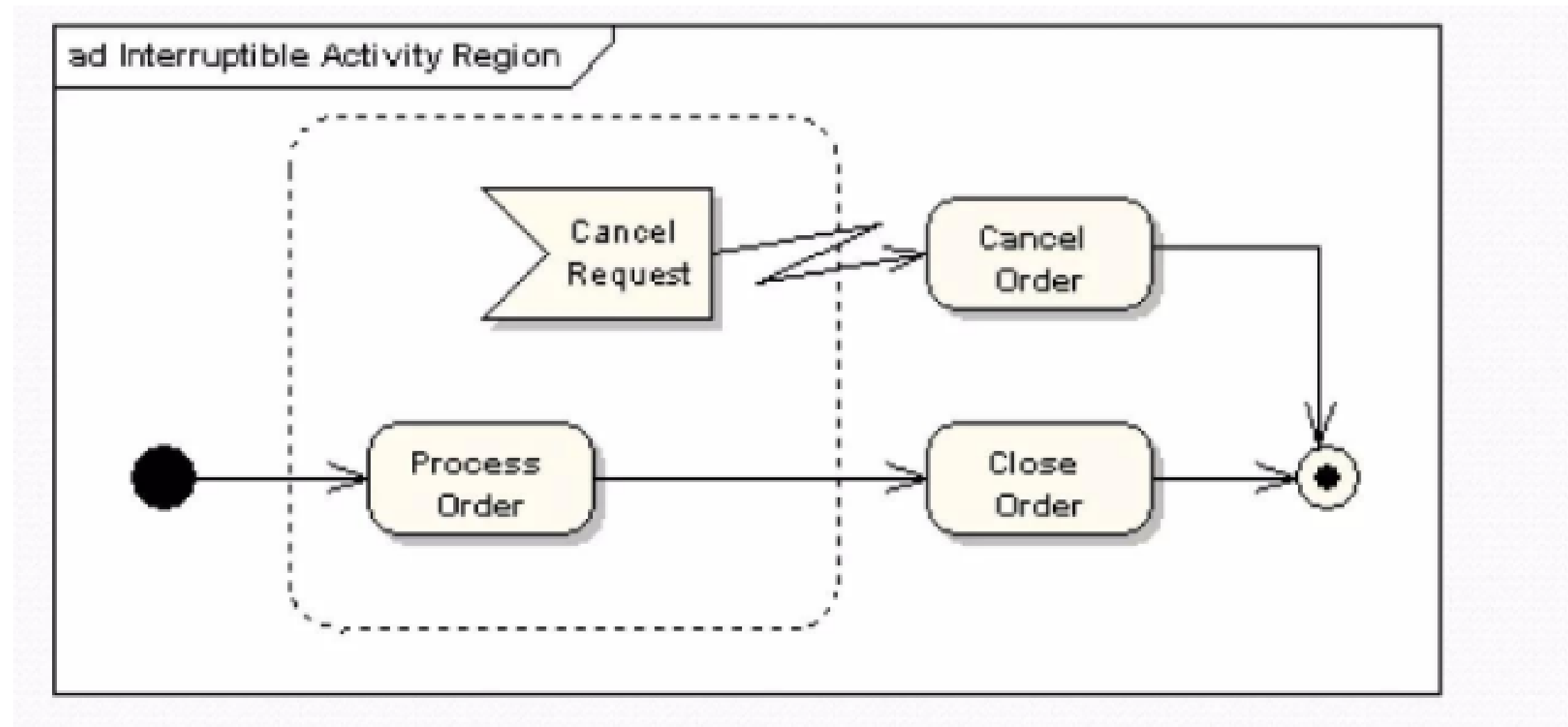
INTERRUPTIBLE REGION

These are useful when an ongoing process can be stopped or canceled.

Sometimes, an entire sequence of activities can be cancelled. Think of placing an order online. You can cancel it while it's 'processing'. We model this with an Interruptible Activity Region, denoted by a dashed rounded rectangle

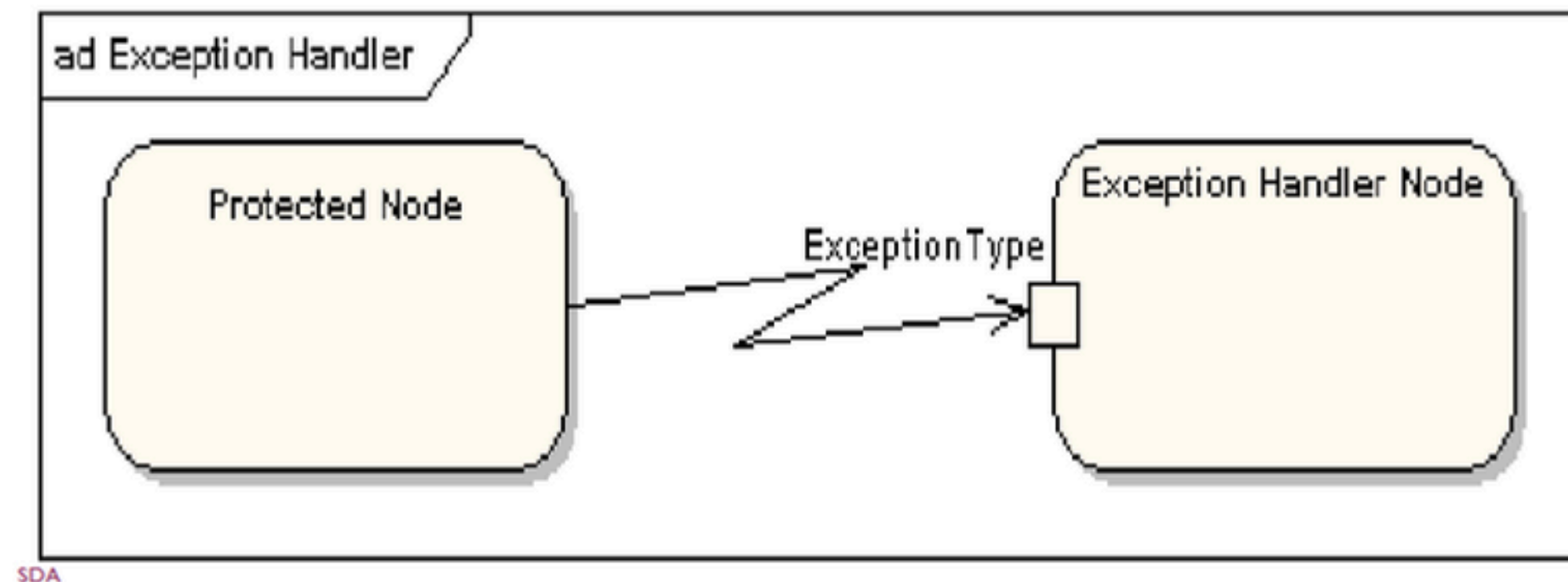
If an interrupting event (like 'Cancel Request') occurs anywhere inside this region, all activities and tokens within the region are immediately terminated, and the flow follows the interrupting edge.

INTERRUPTIBLE REGION



EXCEPTION HANDLING

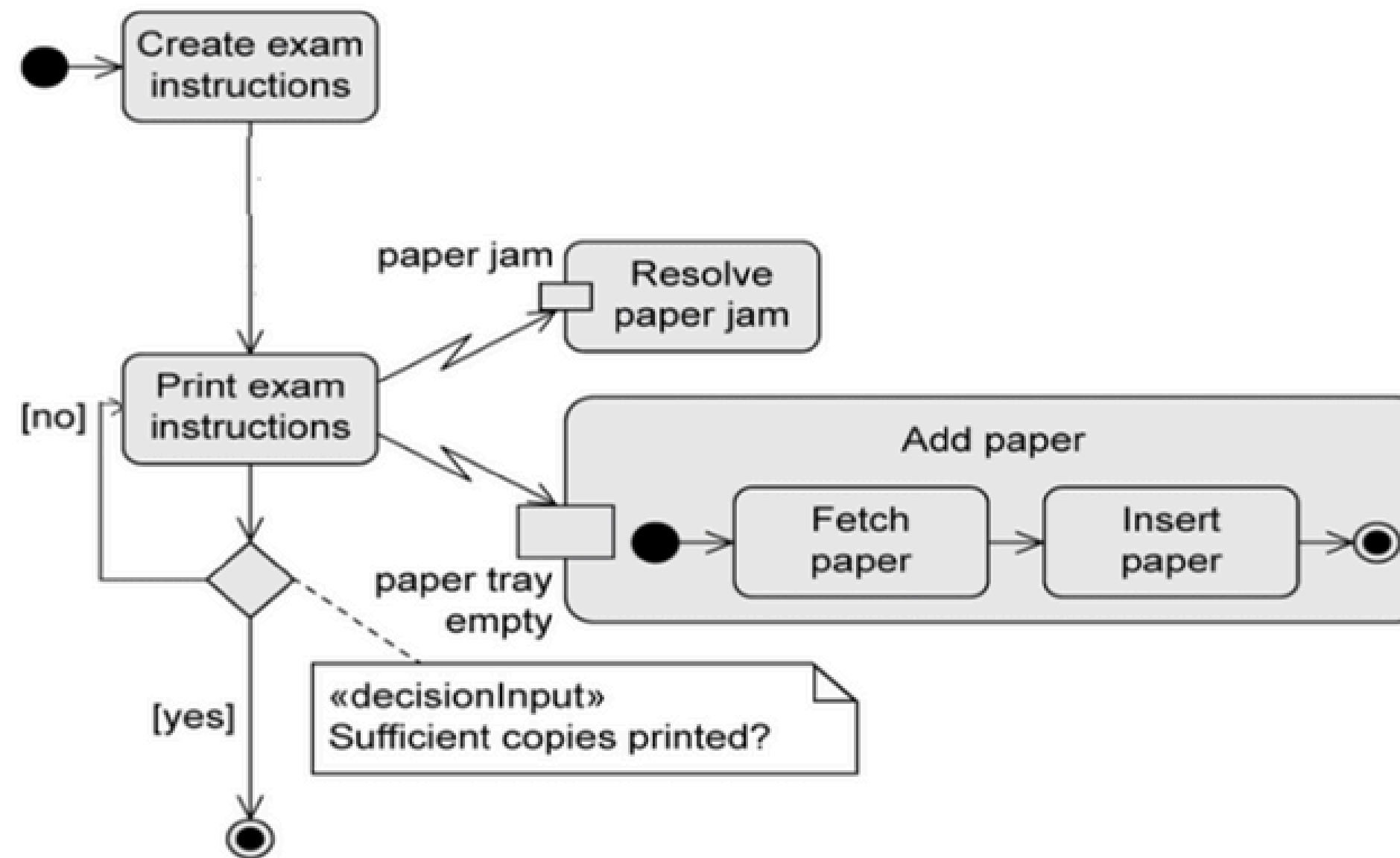
A Protected Node is an activity that might throw an exception. The Exception Handler Node is the activity that deals with a specific type of exception if it occurs. It's like a try-catch block in code.



EXCEPTION HANDLING

Example: The 'Print Exam Instructions' example is perfect. The protected node is Print exam instructions. If a paper jam occurs, the handler Resolve paper jam is executed. Afterward, the flow can merge back and check [Sufficient copies printed?] to continue normally.

EXCEPTION HANDLING



SENDING AND ACCEPTING SIGNALS

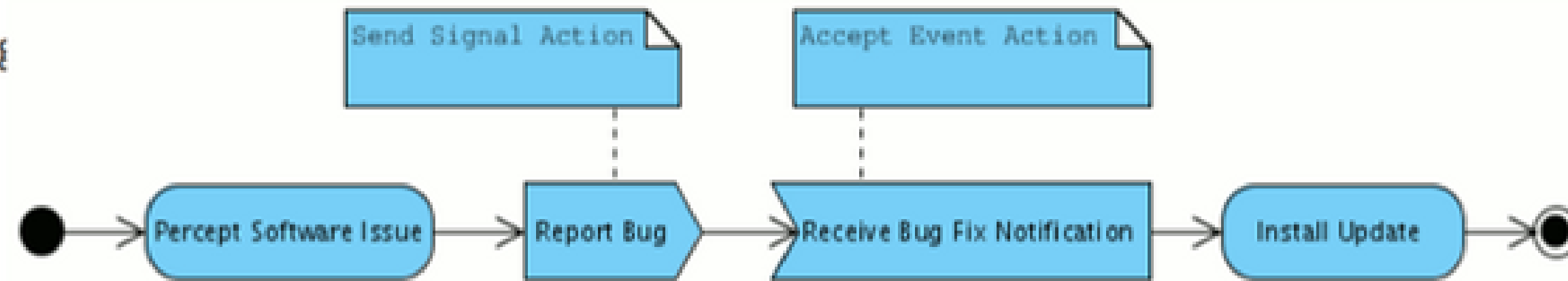
These show asynchronous communication with the outside world.

- Send Signal Action: A convex pentagon. It sends a signal and immediately continues its flow.
- Accept Event Action: A concave pentagon. It waits for a signal to arrive before proceeding.

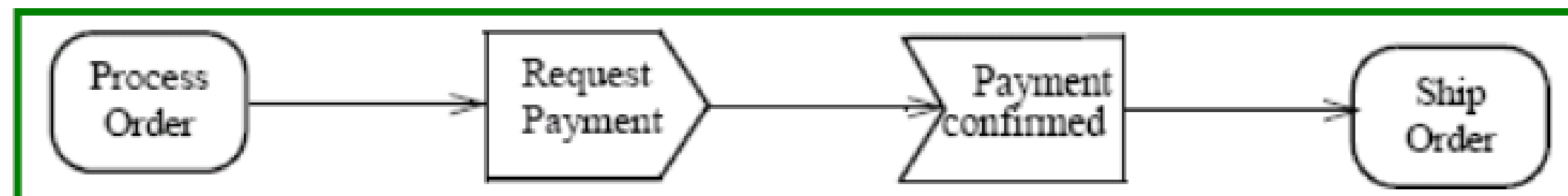


SENDING AND ACCEPTING SIGNALS

In the following

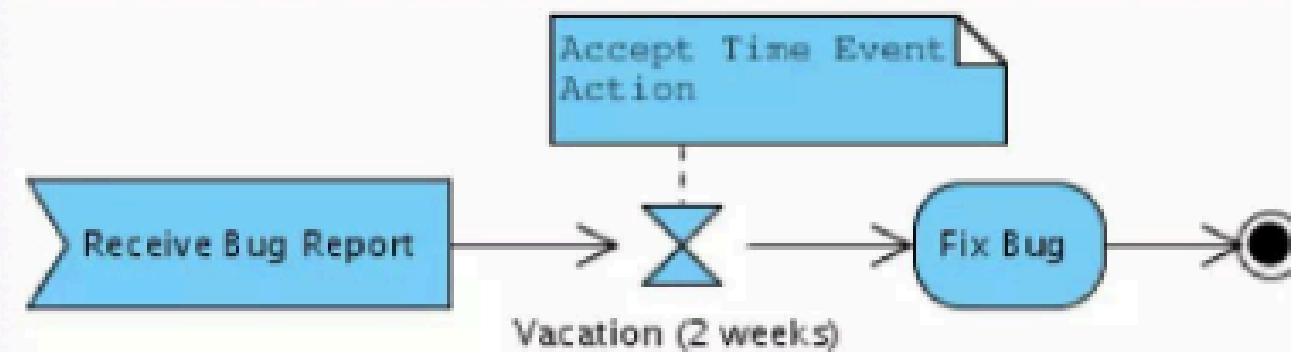


Note that the activity flow gets interrupted - gets into a wait state - until the bug fix notification is received. (If there was no receive signal action, however, the flow would just continue after executing the send signal action.)

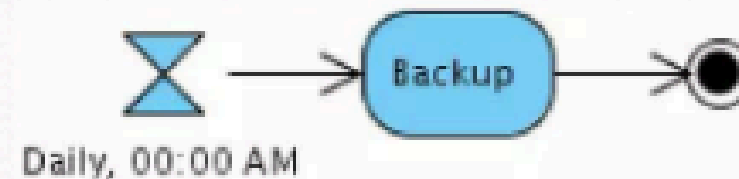


TIME EVENTS

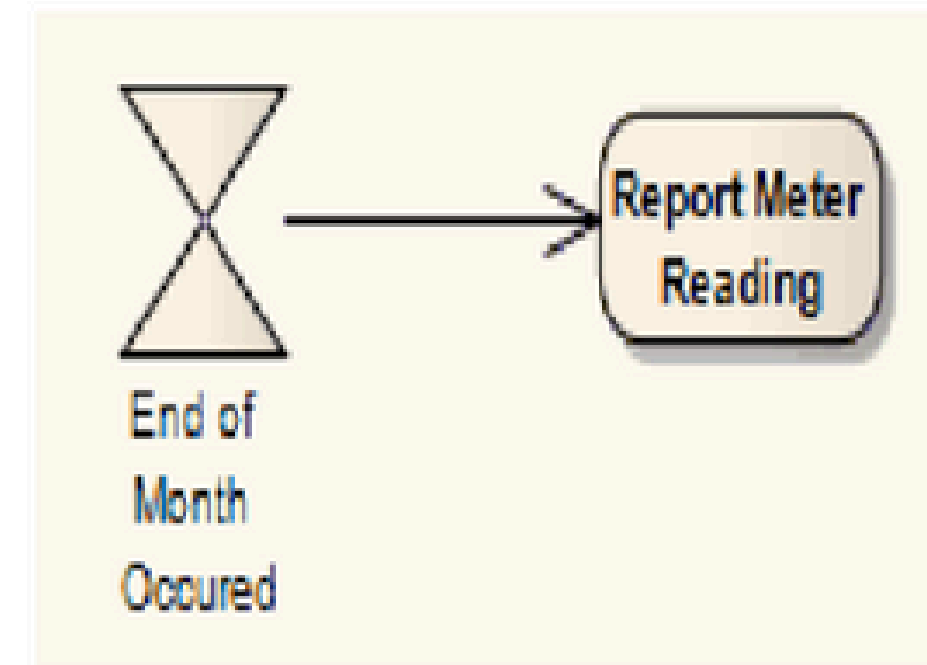
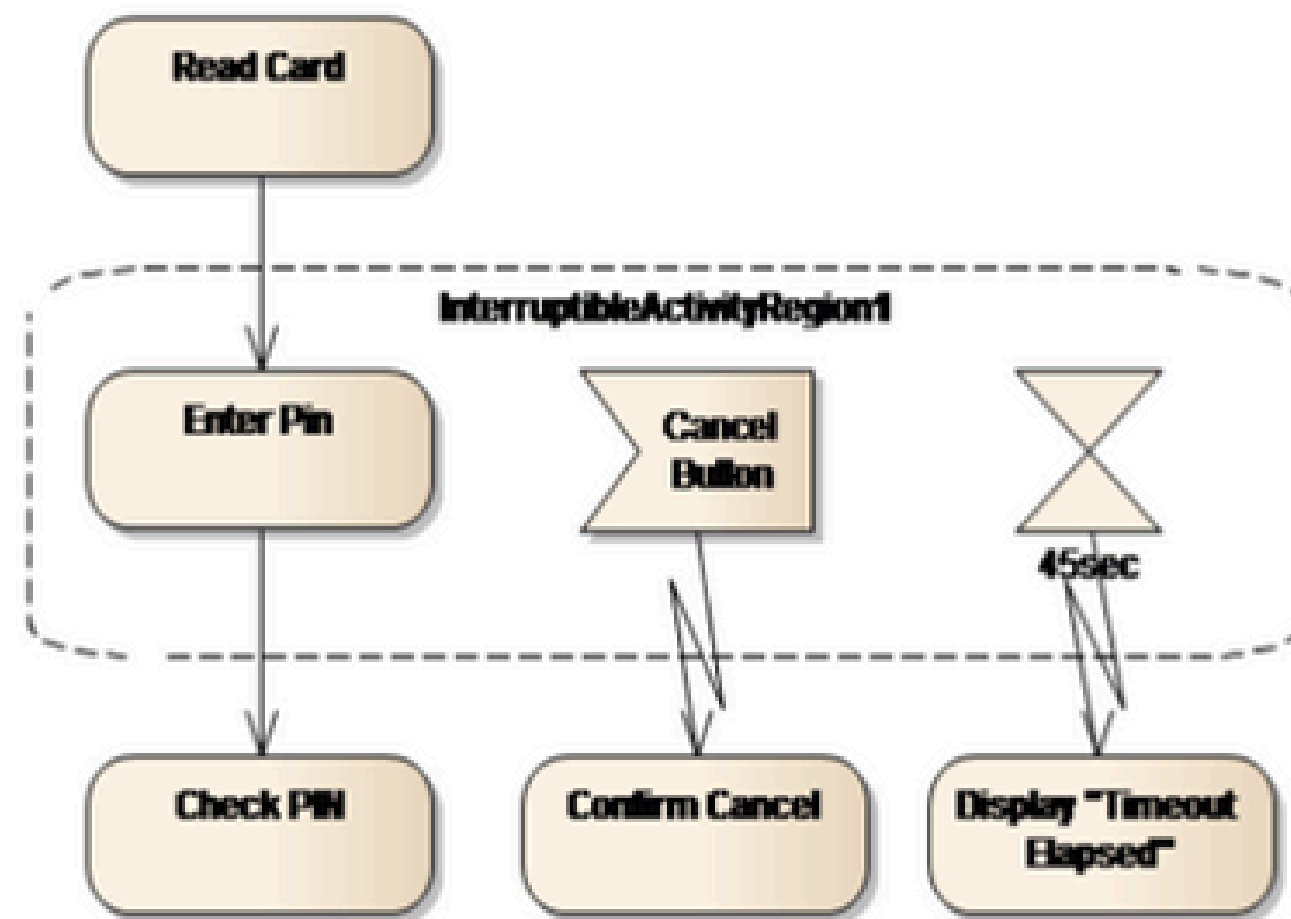
A time event models a wait period:



An activity starting with a time event is launched periodically:



TIME EVENTS



ELECTRONIC PRESCRIPTIONS

ACTIVITY DIAGRAM

The Electronic Prescription Service enables prescribers - such as general practitioners (GPs) and practice nurses - to send prescriptions electronically to a dispenser (such as a pharmacy) of the patient's choice.

As with all NHS Connecting for Health services, access to the EPS is controlled through the use of the NHS Smartcard having user's name, photograph and unique identity number printed on, and with embedded smart chip. The Smartcard gives individual users different levels of access depending on their role.

Prescriber logs onto the clinical system using their Smartcard and passcode, chooses medication or medical appliance for the patient, adds prescribing endorsements where required, and applies electronic signature to authorize the electronic prescription. Electronic prescription is transmitted to the EPS. Prescription token is printed where required. Authorized person hands prescription token to patient where necessary.

Paper copies of electronic prescriptions are called tokens. They act as a hard copy of the details contained within the electronic prescription. There are two types of tokens - 'prescription tokens' and 'dispensing tokens'. When the EPS is fully introduced, paper copies of electronic prescriptions will no longer be needed but should still be available by request, whenever necessary.

